

Spec-Driven Development



References

- 01 O'Reilly, *AI-Native Software Engineering* (early release), ch. 5–6.
- 02 Anthropic, “The AI-native SDLC playbook.” claude.com/blog/the-ai-native-sdlc-playbook
- 03 IEEE Std 610.12-1990, *Standard Glossary of Software Engineering Terminology*.
- 04 ISO/IEC/IEEE 29148:2018, *Systems and software engineering: Requirements engineering*.
- 05 A. Mavin et al., “Easy Approach to Requirements Syntax (EARS),” IEEE RE’09, 2009.
- 06 Prof. Richard Jullig, UC Santa Cruz.
- 07 D. Song et al., “An Empirical Study of Code Generation Errors made by Large Language Models,” MAPS ’23, 2023.
- 08 Q. Chen et al., “A Deep Dive Into Large Language Model Code Generation Mistakes: What and Why?” arXiv:2411.01414, 2024.
- 09 GitHub Spec Kit. github.com/github/spec-kit
- 10 Star History, github.com/spec-kit. star-history.com
- 11 Kiro (AWS), OpenSpec, BMAD Method and Tessel project sites.
- 12 M. Pocock, “grill-me” skill.

What we covered last week

01

Engineering ≠ programming

Coding shrinks. Requirements, design, validation and change matter more.

02

Vibe coding → engineering

What separates them isn't using AI. It's how the output gets verified.

03

More specifying, less typing

When the agent gets it wrong, fix the gap in the spec. Don't rephrase and hope.

04

Five AI-native patterns

Shift up to the why, shift left on intent, and design the environment for agents.

Team formation: where we are

8

teams with members

32

students in a team

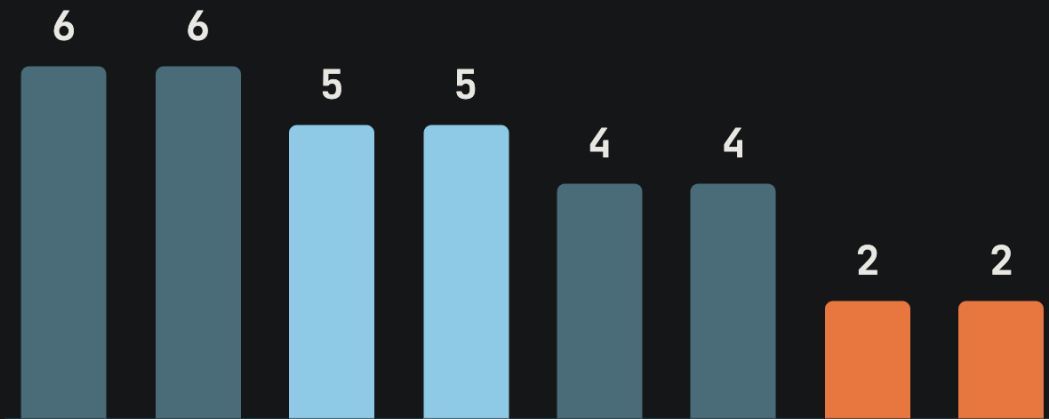
2

registered, no team yet

20

not on GrepThink yet

TEAM SIZES • TARGET 5



THIS WEEK

- Not on GrepThink? Register now.
- No team? Join one with open spots.
- Teams of 2: recruit up to 5.
- 3 projects have no members: merge or delete duplicates.

Today

01

The AI-native software development life cycle

02

What is spec-driven development?

03

SDD and human-in-the-loop

04

Plan Mode: the bridge

05

Three levels of maturity

06

What good specs look like

07

The SDD workflow

08

SDD frameworks

LAB

Lab 2: Spec Kit in practice

01 The AI-native Software development life cycle

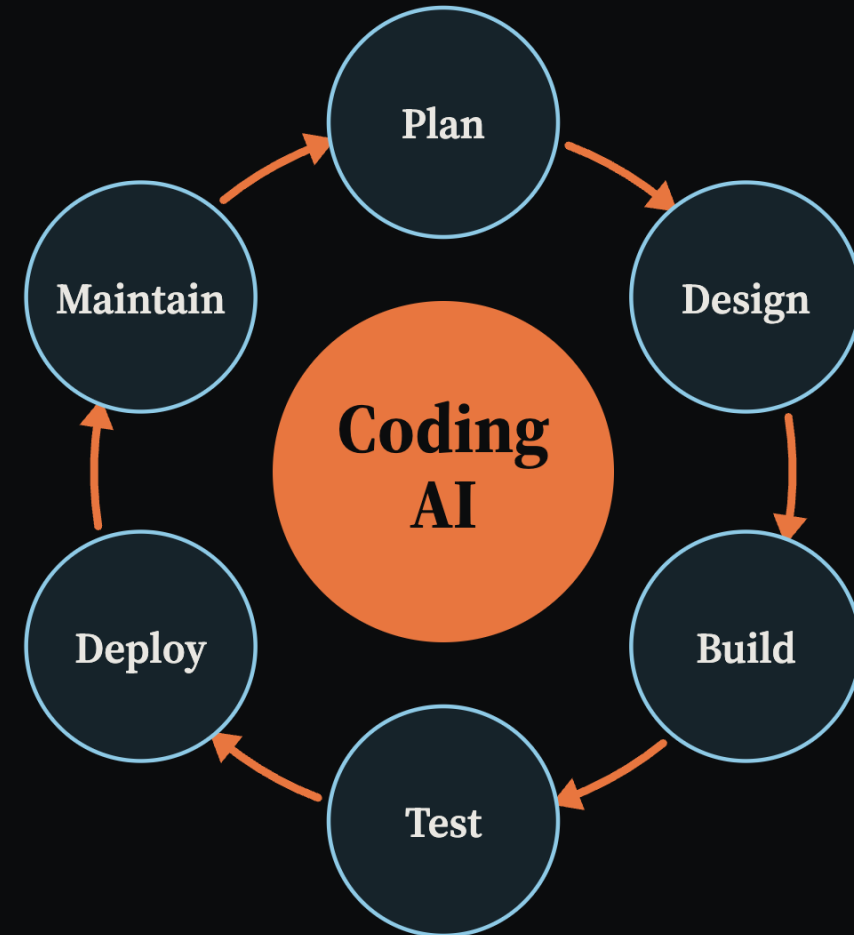
What is the SDLC?

The software development life cycle (SDLC) is the process that takes software from idea to production.

Traditional: the line



AI-native: **the loop**

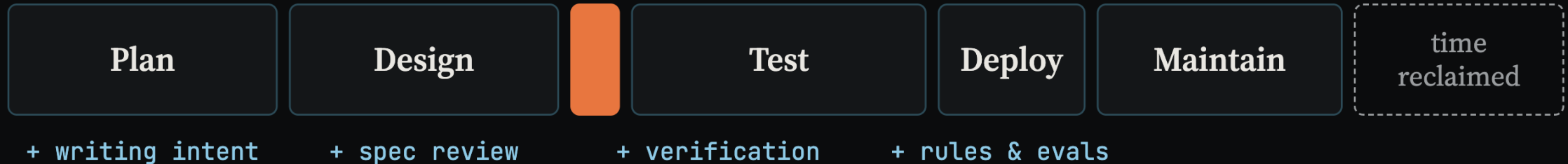


Build shrinks. The work around it grows.

Before agents: every stage runs at human speed



With agents: build runs at agent speed, the rest takes more care



Code is no longer the bottleneck

The bottleneck moves

Planning, review and deployment now set the pace, not writing code.

Human-speed controls stall

Approval steps built for weekly releases can't keep up with hourly changes.

Governance costs spike

Every exception that goes to a committee becomes a queue.

Six stages, one artifact chain

1 Plan

Agent turns an idea into intent.md. The originator approves it.

2 Design

Agent writes spec.md with org rules applied. Product owner signs off.

3 Build

Agent writes plan.md before code. The engineer interrogates the plan.

4 Test

Agent iterates against checks it can run itself. Tests stay locked.

5 Deploy

Agent pre-reviews the PR. Humans judge intent and risk, and approve.

6 Maintain

Alerts trigger diagnosis. A fix starts as a new intent.md.

1 Plan: capture intent

BEFORE

An idea goes through the backlog, user stories, story points and refinement meetings. Each handoff loses some of what was meant.

NOW

The person with the idea brainstorms with an AI and the team and saves **intent.md**: what, why and constraints. The product owner reviews it before it's committed.

```
# Intent: claims status self-service
## Problem
Customers call to ask where their
claim is. About 1/3 of call time.
## Proposed outcome
Status, next step and date in the portal
## Affected users and systems
Claims handlers, portal, claims API
## Constraints
No new personal data. Existing login.
## Open questions
Do outside loss adjusters need access?
```

2 Design: requirements + design

BEFORE

Analysts write the requirements, then designers turn them into a design. Two teams and two handoffs: slow, and details get lost.

NOW

One session: turn `intent.md` into **`spec.md`** with AI using the team's brand, security and UX rules, and flags concerns. The product owner reviews it.

THE PROMPT

Read the attached `intent.md` and write a requirements and design spec for our existing codebase.

Apply our skills for brand, security and UX standards.

Save it as `spec.md` for engineering. Flag any areas of concern, especially where policies contradict.

3 Build: plan before code

BEFORE

Engineers code from tickets. Team knowledge lives in people's heads, so rework shows up late in review.

NOW

The agent interviews the engineer, writes **plan.md**, then builds from it with project rules, skills and hooks loaded.

WHAT THE AGENT READS

<code>spec.md</code>	what to build
<code>plan.md</code>	how, file by file
<code>AGENTS.md</code>	conventions, commands
<code>skills/</code>	team policies
<code>hooks</code>	hard guardrails

Independent tasks can run
in parallel sessions.

4 Test: build in the feedback loop

BEFORE

QA tests at the end. Bugs show up after the code is already called done.

NOW

Testing is part of the build. The agent loops until a clear target passes. Bug fixes start with a **locked** failing test.

DONE MEANS

- [x] all tests pass
- [x] types and lint clean
- [x] screenshot matches the mock

Bug fix:

1. write the failing test
2. lock the test file
3. agent fixes the code
4. test goes green

5 Deploy: review and gated release

BEFORE

People review every line by hand, and releases wait in approval queues.

NOW

AI pre-reviews each PR against policy; people judge intent and risk. A **named person** approves production.

WHO MAY DO WHAT

dev	agent deploys freely
PR	AI review + code owner
production	a named human approves

The agent never approves its own work.

Every allow / block / ask decision is logged.

6 Maintain: close the loop

BEFORE

Someone notices a problem, files a ticket, and it waits in a queue.

NOW

Alerts trigger the agent. It diagnoses read-only and proposes a fix as a PR. The fix starts as a new **intent.md**.

RESPONSE TIERS

small spike	→ log it
bigger	→ diagnose, read-only
severe	→ propose a fix as a PR

Then: new `intent.md`
→ back to Plan

Put team knowledge where the agent can read it

Project context file

Conventions, build and test commands, recurring mistakes. Under a page. Update it when the agent repeats an error.

Skills

Team policies like security, API and brand rules. Versioned, owned by the policy owner. Advisory.

Hooks

Scripts that block, allow or log an action. Use them for rules that must hold with no exceptions.

Autonomy goes down as risk goes up

Where	What the agent may do
Development	Act freely in a sandbox with scoped credentials.
Pull request	Review against policy and fix comments. A code owner approves.
Production	Nothing without a named human's approval. It never approves its own work.
Incidents	Small deviation: log. Larger: diagnose read-only. Severe: propose a fix as a PR.

02 What is spec-driven development?

What is a user story?

As a [type of user], I want [goal] so that [reason].

“As a new user, I want to create an account so that I can see my personal dashboard.”

“As a returning user, I want to log in securely so that I can pick up where I left off.”

✓ Focused on the outcome ✓ No technology ✗ Too vague to build from

What are software requirements?

“A condition or capability needed by a user to solve a problem or achieve an objective.”

IEEE DEFINITION

Requirements say **what** the system must do. Never **how**.

Example requirements: login

- The system must allow users to register with an email and password.
- Passwords must meet minimum security standards.
- The system must prevent duplicate accounts for the same email.
- The system must reject invalid or unrecognized credentials.
- The system must support password recovery.
- The system must lock accounts after repeated failed login attempts.

Still technology-agnostic: you know **what** to build, not yet **how**.

Functional vs. non-functional

FUNCTIONAL

What the system does

- Register with email and password
- Log in returning users
- Reject invalid credentials
- Lock accounts after 5 failures
- Support password recovery

NON-FUNCTIONAL

How well it does it

- Login responds in under 300 ms
- Rate limit: 10 requests/min per IP
- Availability: 99.9% uptime
- Handles 10,000 concurrent logins
- Passwords stored with strong hashing

What are software specifications?

A software specification is a written description of what a system must do and how well it must do it, agreed on before anyone builds it.

Examples of software specs

API SPEC

Endpoints

POST /auth/register returns 201, or 409 if the email exists.

DATA SPEC

Data model

users: id, email (unique), password_hash, locked_until

UI SPEC

Screens

Login form with email, password and a “Forgot password?” link.

FEATURE SPEC

spec.md

Goal, constraints, acceptance criteria, out of scope.

DESIGN DOC

Technical design

Current state, proposed design, trade-offs, rollout plan.

TEST SPEC

Acceptance tests

Given 5 failed logins in 15 minutes, the account locks.

So what is spec-driven development?

Spec-driven development (SDD) is a way of building software where a written specification, not the code, is the **source of truth**. You define what and why first; then AI agents plan, build and verify the code against that spec.



“Fix the login bug.”

HUMAN

Fills the gaps from shared context

- Knows the product and the team
- Remembers last week's bug
- Asks: which flow? which browser?
- Notices when a fix looks wrong

AGENT

Fills the gaps with guesses

- Starts every session from zero
- Doesn't know your team or users
- Rarely asks; picks the typical answer
- Moves fast, so a wrong guess spreads

The spec gives the agent the context a teammate already has in their head.

Why ambiguity costs more with agents

VELOCITY

50 → 500 lines

While a developer writes 50 lines on a wrong assumption, an agent writes 500, all built on the same guess.

WHY Each token builds on the ones before, so an early guess carries through.

SILENCE

No questions

A human stops and asks. An agent resolves the ambiguity on its own and keeps going.

WHY It predicts the most likely answer, not yours.

Why models behave this way

01

Most likely, not yours

A model predicts the most probable next token. A vague prompt gets the typical answer from its training data.

02

Context is its only memory

Your team, users and yesterday's meeting don't exist unless they're in the prompt or the files.

03

Trained to finish, not ask

Training rewards completing the task, so it answers instead of asking a clarifying question.

04

Early choices compound

Each step builds on the last. It rarely goes back to question an early assumption.

Garbage in, clean-looking garbage out

It compiles and runs

The error is in the meaning, not the syntax. Shallow tests still pass.

WHY It learned from huge amounts of working code, so the output always looks fluent.

Research agrees

Song et al. (2023): many errors come from requirements missing in the prompt.

WHY Missing requirements get filled with typical ones.

Errors compound

One early wrong guess, like sync instead of async, ends up under 500 reasonable-looking lines.

WHY Each step builds on the last, and it writes tests for its own guess, so they pass.

“Add authentication to the admin panel.”

Open question	If you don't decide, the agent picks...
Which login: OAuth2, JWT, sessions or magic link?	Whatever is most common in tutorials
How long do tokens last? Refresh tokens?	The library default
Token expires mid-session: redirect or error?	Redirect to login, work lost
Not logged in: 401, 403, or both?	One code, used everywhere
What happens to today's sessions?	Not handled
“Stay logged in”? For how long?	Not built, or forever

User story vs. requirements vs. spec

	User story	Requirements	Spec
Written by	PM	PM + engineer	Engineer
Language	User language	Business / system	Technical
Answers	Why & what (user)	What the system must do	What & how (engineering)
Can an agent use it?	✗ Too vague	◐ Partially	✓ Agent fuel

DISCUSS How much spec do we need before we start building?

Agents consume specs,
not user stories.

03 SDD and human-in-the-loop

Four human checkpoints

BEFORE PLANNING

Spec review

Highest leverage: minutes to fix here, hours once it's code.

BEFORE EXECUTION

Plan review

Review the plan and make sure that it follows the specs and the requirements

DURING EXECUTION

Code review

Does the code solution meet the criteria and fit the codebase?

BEFORE PRODUCTION

Deployment review

Monitor during the deployment and test right after.

Tell the agent when to stop and ask

Trigger	Stop and ask when...
Ambiguity	Two valid options; it can't pick.
Divergence	A test still fails after 3 tries.
Contradiction	The code contradicts the criteria.
Irreversibility	About to delete or migrate real data.

Let the agent interview you first

I want to build [feature].

Before writing anything, ask me as many questions as you need for a complete spec.md: goal, constraints, acceptance criteria, non-goals and edge cases.

Don't write the spec until I say so.

Answers replace guesses

Without the interview, the agent fills silences with what most teams do.

Make it a team activity

PM, designer and engineer answer the questions together.

04 Plan Mode: the bridge

Plan Mode: plan first, then build

```
> Add pagination to GET /api/orders.  
The frontend expects cursors.
```

PLAN

1. routes/orders.ts: accept cursor, limit
2. orderService.ts: WHERE on cursor, not OFFSET
3. Response adds nextCursor, hasMore
4. Update types in types/api.ts
5. Tests in tests/routes/orders.test.ts

```
? Cursor on order ID or createdAt?  
Assuming ID.
```

Read-only until you approve

Claude Code (Shift+Tab), Cursor and other agents.

Review it like a PR

What's missing? Here: the API docs.
Its question is a gap in your spec.

Plan Mode is a bridge, not SDD

It's transient

The plan lives in one chat. A teammate can't pick it up tomorrow.

It answers how, not what

It takes your description at face value and won't surface missing requirements.

1

Plan Mode on every non-trivial task



2

Notice what you have to correct



3

Write those things in spec.md first



4

Use Plan Mode to check it read the spec

05 Three levels of maturity

Three levels of SDD maturity

LEVEL 1

Spec-first

Write the spec before the agent starts: goal, constraints, acceptance criteria.

Use for any task over 30 minutes of agent work.

LEVEL 2

Spec-anchored

The spec lives in the repo and changes in the same PR, updated first.

Use for code that keeps changing.

LEVEL 3

Spec-as-source

People edit only the spec; code is generated from it. Still theoretical at scale.

Watch it, don't adopt it yet.

Two tiers of specs

STRATEGIC

Rules file · Architecture Decision Records (ADRs) · system requirements

Living documents. Updated when the system's direction changes; every agent session reads them.

TACTICAL

spec.md · plan.md · tasks.md per change

Drive one feature or fix, then stay as history. A follow-on feature gets a new spec.

06 What good specs look like

Start every spec from four headings

GOAL

What does success look like?

CONSTRAINTS

What must the implementation respect?

ACCEPTANCE CRITERIA

How will you verify it's done?

OUT OF SCOPE

What should the agent not touch?

Same request, two specs

BAD

“Add a caching layer to improve performance.”

The agent chooses the library, TTL, cache key, invalidation and monitoring for you.

Goal

Cache products by category ID

Constraints

- Existing node-cache; no new infra
- TTL (time to live): 5 minutes
- Invalidate on product.updated

Acceptance criteria

- 1st request hits the DB, then cache
- Invalidated within 1 second
- Hit/miss ratio on dashboards

Out of scope

- Distributed cache, other endpoints

Beyond the happy path

WHAT IF IT BREAKS?

Failure mode

If the cache is down, requests go straight to the DB.

WHAT IF TWO THINGS HAPPEN AT ONCE?

Concurrency

Two updates at once never leave old data in the cache.

HOW DO WE KNOW IT WORKS?

Observability

The hit/miss ratio shows up on the dashboard.

HOW DO WE UNDO IT?

Rollback

A config switch turns caching off within a minute.

“Add a search bar to the products page.”
Write the requirements.

FUNCTIONAL • 3

What the system does

What can the user search, and what do they get back?

NON-FUNCTIONAL • 3

How well it does it

How fast, how many users, which devices? Give numbers.

07 The SDD workflow

One folder per change, three core files

```
specs/  
  001-notification-prefs/  
    spec.md  
    plan.md  
    tasks.md  
    risks.md  
  002-payment-retry/  
    ...
```

spec.md

What and why. No file paths or tech choices.

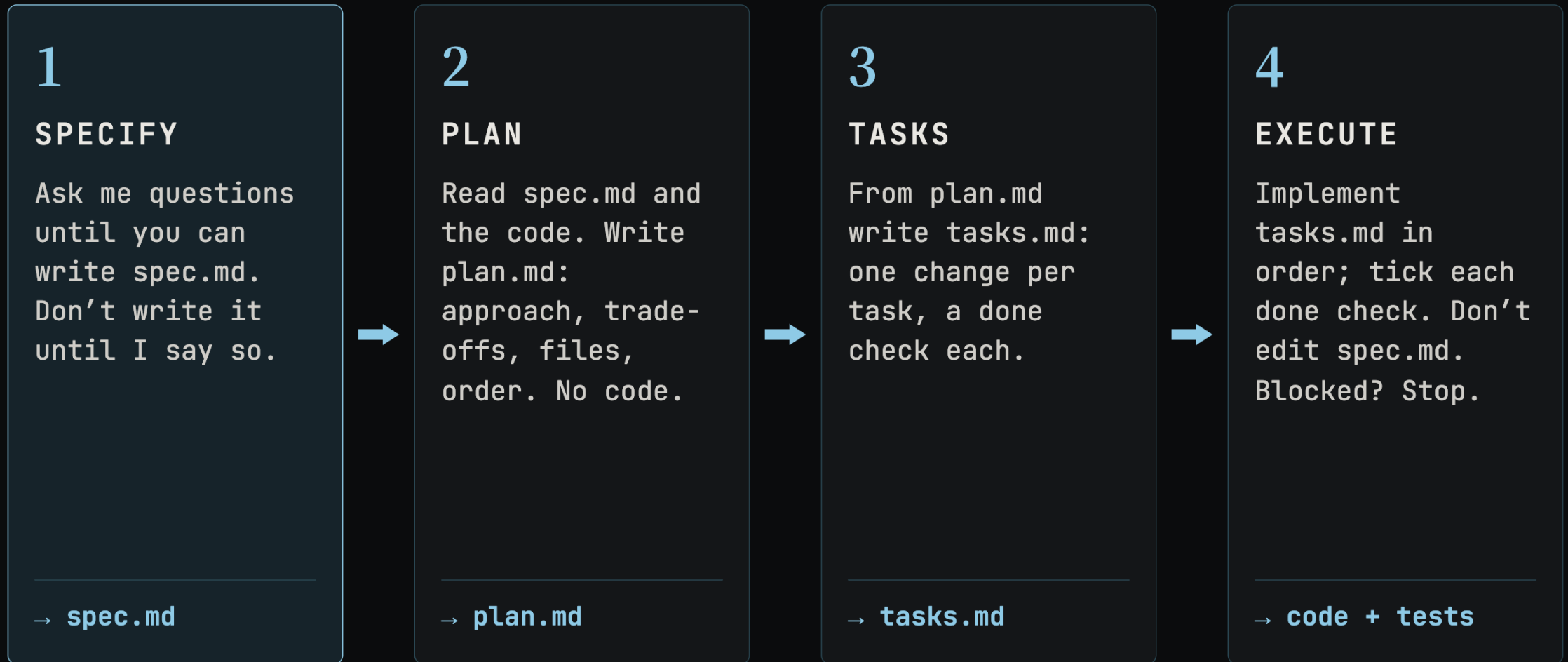
plan.md

How: approach, trade-offs, files, order.

tasks.md

Small tasks, each with a done check.

One short prompt per phase



Every task needs a done check

- [] `PATCH /api/users/me/notifications`
Done when: integration test for valid update, invalid keys and schema validation passes.
- [] `NotificationService.send()`
checks preferences.
Done when: email tests pass and a disabled type is not sent.

No finish line

Without a done check, the agent stops too early, or never stops.

Split megatasks

If the done check says “and” more than once, split the task.

08 SDD frameworks

SDD frameworks at a glance

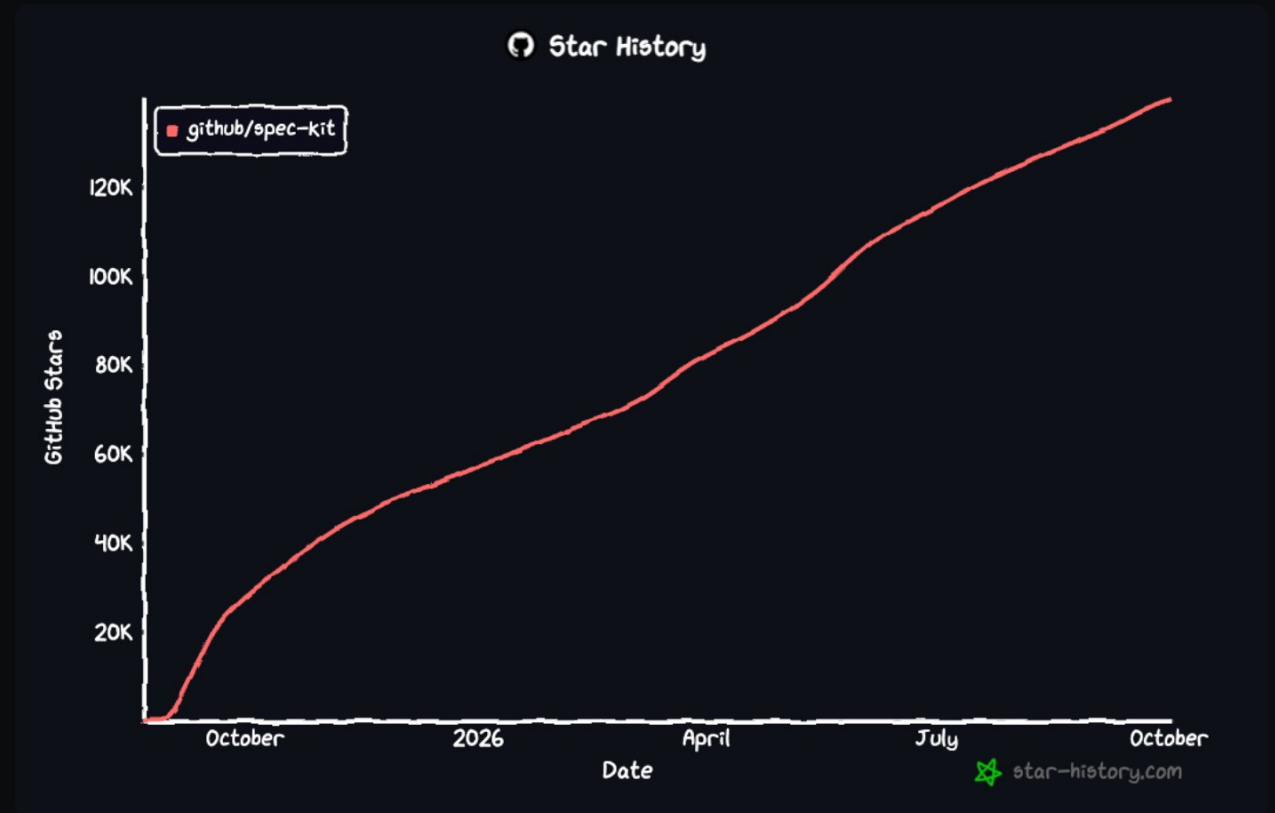
Framework	Workflow	Best for
GitHub Spec Kit	Constitution → specify → plan → tasks → implement	Open source; works with most coding agents
Kiro (AWS)	Requirements (EARS) → design → tasks	An IDE with specs built in
OpenSpec	Propose a change → apply → archive	Lightweight work on existing code
BMAD Method	Analyst, PM, architect and developer agents	Heavy, simulates a whole team
Tessl	One spec per code file; code is generated	Experiments with spec-as-source

GitHub Spec Kit



Define **what** and **why** before deciding **how** to build it.

Constitution once per project; specify → plan → tasks → implement → converge per feature.



Spec Kit Skills

ONCE PER PROJECT

/speckit-constitution

Create principles focused on code quality and testing.

PER FEATURE

/speckit-specify

Build a photo organizer with albums grouped by date.

/speckit-plan

Use Vite with vanilla JavaScript. Store metadata in SQLite.

/speckit-tasks

No input needed

/speckit-implement

No input needed

/speckit-converge

No input needed

Lab 2: Spec Kit, about 45 minutes

1

Set up

Install specify-cli, clone repo-analysis-sprint, run specify init.

2

Constitution

Strict TypeScript, Vitest tests, lint and test before every PR.

3

Specify, plan, tasks

Turn the risk-badge story into spec.md, plan.md, tasks.md.

4

Implement and verify

Lint and tests pass. Open a PR that links your specs/ folder.

Three things to remember

01

The substrate changed. The discipline didn't.

Clear intent, testable criteria, step-by-step verification.

02

Speed amplifies everything.

A good spec gets built fast.
So does a bad one.

03

Match the ceremony to the risk.

Plan Mode for small tasks;
a living spec for features that last.

Questions?

Next week:
Anatomy of Coding Agents

